Axiomander Guidance: Exceptions as Values Rather Than Special Hoare Logic

Executive Summary

Axiomander should not build a dedicated "exception Hoare logic".

Instead, exceptions should be represented as ordinary outcomes of computation.

This follows the direction taken by modern effect systems, monadic verification systems, F*, Iris-derived calculi, and many contemporary semantic formulations.

The core idea is simple:

Instead of modelling a function as producing only a value,

```
A
```

model it as producing:

```
Outcome[A]
```

where:

```
Outcome[A] :=
    Return(A)
  | Raise(Exception)
```

Hoare logic and weakest preconditions then reason about outcomes rather than ordinary values.

This approach scales naturally to databases, transactions, cancellation, concurrency, timeouts, and future effect systems.

Motivation

Traditional Hoare logic assumes only one successful exit:

```
{P} c {Q}
```

This is inadequate for Python because functions may:

- return normally
- raise exceptions
- fail due to runtime errors
- abort transactions
- be cancelled
- time out

Traditional exception calculi introduce separate exceptional postconditions:

```
{P}
c
{Q normal}
{R exceptional}
```

While workable, this becomes increasingly complex as more effect types are introduced.

Instead, Axiomander should treat all exits uniformly.

Core Outcome Type

Internally the verifier should define:

```
Outcome[A] :=
    Return(A)
  | Raise(Exception)
```

Later extensions may include:

```
Outcome[A] :=
    Return(A)
  | Raise(Exception)
  | Cancelled
  | Timeout
  | Abort(TransactionError)
```

The verifier should be designed so that additional outcome variants can be added without changing the core proof infrastructure.

Weakest Preconditions

Traditional WP:

```
WP(c, Postcondition)
```

Axiomander WP:

```
WP(c, Φ)
```

where:

```
Φ : Outcome -> Prop
```

The postcondition is now a predicate over outcomes.

Example:

```
Φ(Return(v)) =
    v > 0

Φ(Raise(NotFound)) =
    user_missing(id)
```

The verifier proves:

```
WP(c, Φ)
```

rather than maintaining separate normal and exceptional proof systems.

User-Facing Contract Syntax

Users should continue writing familiar contracts:

```
"""
requires:
```

```
    user_id > 0

ensures:
    result.id == user_id

raises NotFound:
    missing_user(user_id)
"""
```

Internally this becomes:

```
Φ(Return(v)) =
    v.id == user_id

Φ(Raise(NotFound)) =
    missing_user(user_id)
```

The translation should be invisible to users.

Relationship to Separation Logic

Separation logic should reason about resources independently of outcome type.

Example:

```
"""
requires:
    owns(account)
    * balance(account, b)

ensures:
    owns(account)
    * balance(account, b - amount)

raises InsufficientFunds:
    owns(account)
    * balance(account, b)
"""
```

Internally:

```
Φ(Return(_)) =
    owns(account)
    * balance(account, b - amount)

Φ(Raise(InsufficientFunds)) =
    owns(account)
    * balance(account, b)
```

Ownership appears in both outcomes.

This is a crucial design principle:

> Resources do not disappear merely because control flow changed.

Why This Is Better

Uniformity

Exceptions become ordinary outcomes.

No special proof rules are required.

Extensibility

Future effects become additional outcome constructors.

Examples:

```
Cancelled
Timeout
RetryRequired
TransactionAborted
```

The WP machinery remains unchanged.

Compatibility With Effects

Databases, concurrency, cancellation and distributed systems naturally fit into an outcome-based semantics.

Compatibility With Monadic Verification

Many modern verification systems effectively reason about:

```
Result[A, E]
```

rather than special exception control flow.

The Axiomander outcome model aligns with this tradition.

Internal IR Recommendation

The IR should contain explicit exception-producing nodes:

```
Raise(e)

Try(body, handler)

Return(v)
```

The semantic evaluation relation should produce outcomes:

```
eval : Command -> State -> Outcome
```

rather than assuming normal termination.

Future Extensions

The outcome approach naturally generalises.

Example:

```
Outcome[A] :=
    Return(A)
  | Raise(Exception)
  | Timeout
  | Cancelled
```

Contracts can then describe these states explicitly.

Example:

```
"""
raises Timeout:
    connection_still_valid(conn)

raises Cancelled:
    transaction_rolled_back(db)
"""
```

No changes to WP infrastructure are required.

Recommended Reading

Separation Logic

- Reynolds, Separation Logic: A Logic for Shared Mutable Data Structures
- Software Foundations, Separation Logic Foundations

Exceptions as Values

- Moggi, Notions of Computation and Monads
- Benton & Kennedy, Exceptional Syntax
- Charguéraud, Characteristic Formulae for Higher-Order Imperative Programs

Modern Verification

- Viper Tutorial
- Iris Lecture 3: Basic Hoare Triples

Final Recommendation

Axiomander should model exceptions semantically as values.

The verifier should be built around:

```
Outcome :=
    Return(value)
  | Raise(exception)
```

and weakest preconditions should have the form:

```
WP(command, Outcome -> Prop)
```

User-facing contracts may continue to use familiar syntax such as:

```
requires
ensures
raises
```

but these should compile into outcome predicates internally.

This provides a simpler semantic foundation, better compositionality, and a natural path toward reasoning about transactions, databases, concurrency, cancellation, and future effect systems.